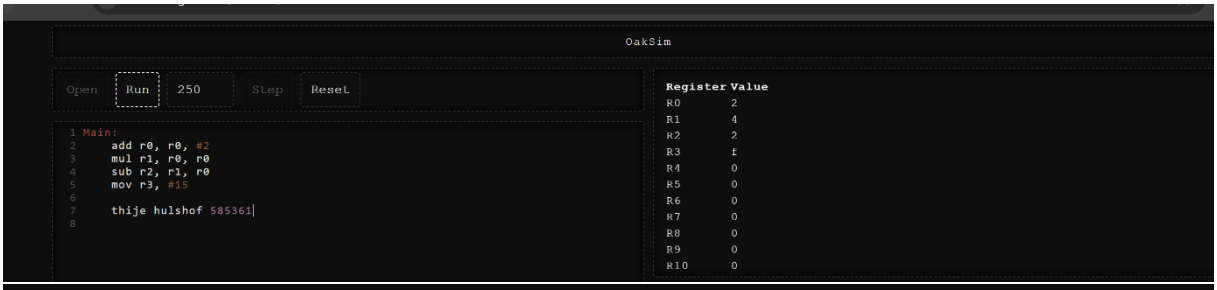
# Template Week 4 – Software

Student number: 585361

### Assignment 4.1: ARM assembly

Screenshot of working assembly code of factorial calculation:



**Assignment 4.2: Programming languages**

Take screenshots that the following commands work:

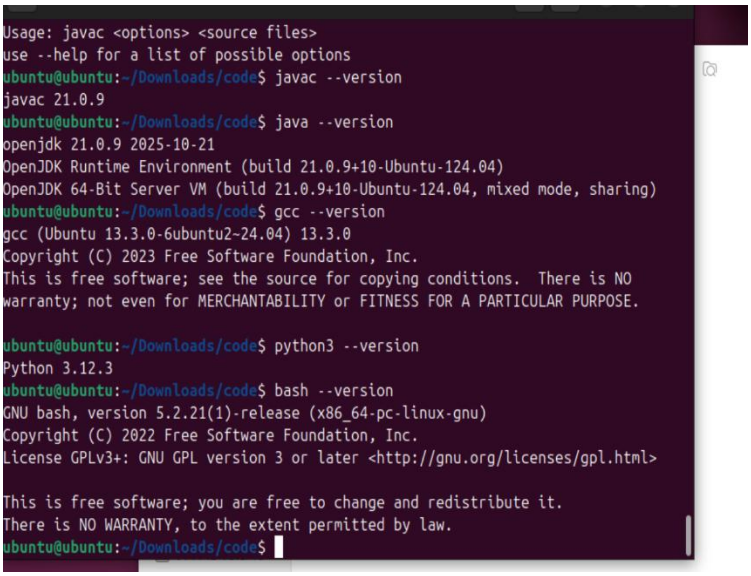
javac --version

java --version

gcc --version

python3 --version

bash –version



```
Usage: javac <options> <source files>
use --help for a list of possible options
ubuntu@ubuntu:~/Downloads/code$ javac --version
javac 21.0.9
ubuntu@ubuntu:~/Downloads/code$ java --version
openjdk 21.0.9 2025-10-21
OpenJDK Runtime Environment (build 21.0.9+10-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.9+10-Ubuntu-124.04, mixed mode, sharing)
ubuntu@ubuntu:~/Downloads/code$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

ubuntu@ubuntu:~/Downloads/code$ python3 --version
Python 3.12.3
ubuntu@ubuntu:~/Downloads/code$ bash --version
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
ubuntu@ubuntu:~/Downloads/code$
```

**Assignment 4.3: Compile**

Which of the above files need to be compiled before you can run them?

Fibonacci en fib.c

Which source code files are compiled into machine code and then directly executable by a processor?

Fib.c wordt meteen omgezet in nullen en eenen

Which source code files are compiled to byte code?

fibonacci

Which source code files are interpreted by an interpreter?

Fib.py

Fib.sh, deze worden regel voor regel uitgevoerd

These source code files will perform the same calculation after compilation/interpretation. Which one is expected to do the calculation the fastest?

Fib.c deze wordt meteen omgezet in machine code en draait daardoor heel snel

How do I run a Java program?

Javac Fibbonacci.java

How do I run a Python program?

Python3 fib.py

How do I run a C program?

Gcc fib.c -o fib en dan voor de run ./fib

How do I run a Bash script?

Bash fib .sh

If I compile the above source code, will a new file be created? If so, which file?

Bij Fibonacci en fib.c wel de andere twee niet.

Take relevant screenshots of the following commands:

- Compile the source files where necessary
- Make them executable
- Run them
- Which (compiled) source code file performs the calculation the fastest?
  gcc was best wel ruim het snelst

```
ubuntu@ubuntu:~/Downloads/code$ javac Fibonacci.java
ubuntu@ubuntu:~/Downloads/code$ java Fibonacci
Fibonacci(18) = 2584
Execution time: 0.15 milliseconds
ubuntu@ubuntu:~/Downloads/code$ python3 fib.py
Fibonacci(18) = 2584
Execution time: 0.64 milliseconds
ubuntu@ubuntu:~/Downloads/code$ gcc fib.c -o fib
ubuntu@ubuntu:~/Downloads/code$ ./fib
Fibonacci(18) = 2584
Execution time: 0.03 milliseconds
ubuntu@ubuntu:~/Downloads/code$ bash fib.sh
Fibonacci(18) = 2584
Excution time 4217 milliseconds
ubuntu@ubuntu:~/Downloads/code$
```

**Assignment 4.4: Optimize**

Take relevant screenshots of the following commands:

a) Figure out which parameters you need to pass to **the gcc** compiler so that the compiler performs a number of optimizations that will ensure that the compiled source code will run faster. **Tip!** The parameters are usually a letter followed by a number. Also read **page 191** of your book, but find a better optimization in the man pages. Please note that Linux is case sensitive.

```
ubuntu@ubuntu:~/Downloads/code$ gcc -O3 fib.c -o fib_fast
```

b) Compile **fib.c** again with the optimization parameters

c) Run the newly compiled program. Is it true that it now performs the calculation faster?

```
Processing triggers for libc-bin (2.39-0ubuntu8.5) ...
ubuntu@ubuntu:~/Downloads/code$ gcc -O3 fib.c -o fib_fast
ubuntu@ubuntu:~/Downloads/code$ ./fib_fast
Fibonacci(18) = 2584
Execution time: 0.00 milliseconds
ubuntu@ubuntu:~/Downloads/code$
```

d) Edit the file **runall.sh**, so you can perform all four calculations in a row using this Bash script. So the (compiled/interpreted) C, Java, Python and Bash versions of Fibonacci one after the other.

```
GNU nano 7.2                    runall.sh *
#!/bin/bash
./fib_fast
java Fibonacci
python3 fib.py
./fib.sh
```

**Assignment 4.5: More ARM Assembly**

Like the factorial example, you can also implement the calculation of a power of 2 in assembly. For example you want to calculate $2^4 = 16$. Use iteration to calculate the result. Store the result in r0.

Complete the code. See the PowerPoint slides of week 4.

Screenshot of the completed code here.

Ready? Save this file and export it as a pdf file with the name: **week4.pdf**